UOW MALAYSIA

PART OF THE UNIVERSITY OF WOLLONGONG AUSTRALIA GLOBAL NETWORK

DATABASE MANAGEMENT SYSTEM REPORT

Subject Title:	Database System
Subject Code:	XDCS1054N
Course Title:	Diploma in Computer Studies
Course Lecturer:	Mr Rodney Roland
Group members:	SHONG QIEN BUO (M44100768) CALVINE JOASH DE SILVA (M44100604) IRWIN TAN (M44100798)
Industry:	Education
Database system :	Student Registration System

1.0 Introduction

Technology has become an essential part of many sectors today, and education is no exception. Schools, colleges, and universities need to manage a large amount of information related to students, courses, lecturers, and academic records. In the past, many institutions relied on manual methods such as paper files or basic record books to store this information. Although these methods were sufficient at the time, they can become inefficient and difficult to maintain when the number of students and academic programs increases.

As educational institutions grow, the need for an organized system to manage academic data becomes more important. Tasks such as student registration, course enrolment, and record management require accurate and well-structured information. When handled manually, these processes may lead to mistakes, missing data, or delays in retrieving important records. Because of this, many institutions now use computerized systems to support their administrative and academic operations.

1.1 Education industry overview

The education sector plays a major role in developing knowledge and skills among individuals in society. It includes different types of institutions such as schools, colleges, universities, and training centres that provide learning opportunities for students at various levels. In the past, many educational institutions relied heavily on traditional teaching methods and paper-based records to manage academic information. However, with the growth of technology and digital learning environments, the education sector has gradually adopted more computerized systems to support both teaching and administrative activities.

As the number of students and courses continues to increase, educational institutions are required to handle a large amount of information. This includes student personal details, course registration, lecturer assignments, class schedules, and examination results. Managing all of this information manually can be difficult and may lead to errors or misplaced records. Because of this, many institutions now use digital systems to store and manage their data more effectively.

Database Management Systems (DBMS) help educational institutions organize and store data in a structured way. With a centralized database, staff members can easily access and update important academic records when needed. This not only improves efficiency in daily operations but also helps ensure that the information stored in the system is accurate and consistent. As technology continues to develop, the use of database systems will remain important in helping educational institutions manage their data and support better learning environments.

Mukul, E., & Büyüközkan, G. (2023). *Digital transformation in education: A systematic review of education 4.0*. **Technological Forecasting and Social Change, 194**, 122664. https://doi.org/10.1016/j.techfore.2023.122664

1.2 Student Registration System (SRS) application and industry support

A Student Registration System is a digital platform used by educational institutions to manage student enrolment and academic records more efficiently. In the past, many institutions handled student registration manually using paper forms and physical files. This process often required a large amount of time and effort from administrative staff and could easily lead to errors, misplaced records, or delays in processing student information. As the number of students increases each year, relying on manual methods becomes less practical for institutions that need to manage large volumes of academic data.

The use of a computerized Student Registration System helps educational institutions organize and manage student information in a more structured and reliable way. Through this system, administrators can record student details, manage course enrolments, assign lecturers, and maintain academic records within a centralized database. This improves the overall efficiency of administrative operations and allows staff to retrieve or update information quickly when needed.

In addition, the system also supports better coordination between different departments within the institution. For example, academic staff can access course registration information, while administrative staff can monitor student records and enrolment status. By integrating these functions into a single system, educational institutions can reduce data redundancy, improve data accuracy, and ensure that important information is securely stored. As a result, Student Registration Systems play an important role in supporting modern educational institutions and improving the management of academic data.

. **Anika, A., & Kumar, D. (2023).** *Streamlining Student Enrolment: University's College Registration System*. Rawat Prakashan. A related chapter similarly describing manual processes as "prone to mistakes" https://lrcdrs.bennett.edu.in/items/6fc43717-5acf-43e4-8e21-ba58d181b9d0

1.3 Project Goals and Objectives

Educational institutions are required to manage large amounts of academic information such as student records, course registrations, lecturer assignments, and academic performance. When these processes are handled manually or through separate systems, it can lead to inefficiencies, data duplication, and difficulties in accessing accurate information. These issues may affect the overall administrative workflow and slow down important processes such as student enrolment and record management. Therefore, implementing a centralized database system can help improve the efficiency and reliability of managing academic data.

1.3.1 Project Goals

The main goal of this project is to design and develop a Student Registration System that stores and manages important academic information within a centralized database. The system focuses on organizing student details, course registration records, lecturer information, and other related academic data in a structured manner. By using a database management system, the project aims to reduce manual work, minimize data redundancy, and ensure that information is stored accurately. In addition, the system will allow administrators to access and manage student registration information more efficiently, which can improve the overall management of academic operations.

1.3.2 Project Objectives

1. To design a database system that stores and manages student registration information in a structured and organized manner.

2. To ensure data accuracy, consistency, and integrity through the use of a centralized database system.

3. To support efficient student registration and course enrolment processes within an educational institution.

4. To enable easy access, updating, and retrieval of student and academic records when need

5. To develop a relational database that manages essential information such as students, courses, lecturers, and registrations.

2.0 Database system in the education industry

Educational institutions such as schools, colleges, and universities need to handle a large amount of data on a regular basis. This includes student information, course details, lecturer allocation for different courses, student enrolment, and student results. As the number of students and courses increases, handling the data becomes more complicated. Therefore, the need for a database system arises to facilitate the handling and management of the data in an effective manner.

Traditionally, educational institutions have been handling their data with the help of a manual system of record-keeping. This includes the use of papers and spreadsheets to keep the data organized and easily retrievable. Although such a system was effective for smaller-scale businesses and organizations, it poses a number of disadvantages for larger-scale businesses and organizations. Manual systems take a long time to manage and update the data. This includes the possibility of errors and the risk of duplication and loss of data. For instance, errors may result in discrepancies in student records. Similarly, the process of retrieving the data from the papers and spreadsheets may become tedious and complicated for a larger-scale business or organization.

To overcome the disadvantages of a traditional manual system of record-keeping, many educational institutions today have started to adopt Database Management Systems (DBMS) for the effective management and maintenance of the data. This includes the advantage of increased accuracy and consistency of the data with the help of validation rules and constraints that can be imposed during the process of data entry into the database management system. This also includes the advantage of increased speed in the process of retrieving the data with the help of a database management system. Similarly, the security of the data is also increased with the help of a database management system, as only authorized persons can access the data with the help of authentication and access control mechanisms.

In the context of a Student Registration System (SRS), a database plays a significant role in managing the process of student enrolment and academic records in educational institutions. The use of a database system can prove to be beneficial in this regard as it can provide a reliable way of handling academic data in a more efficient manner.

Kaka, A. R. (2023, June 17). *Unleashing the potential of cloud databases in education; enhancing collaboration, accessibility*. The Financial Express. https://www.financialexpress.com/jobs-career/education-unleashing-the-potential-of-cloud-databases-in-education-enhancing-collaboration-accessibility-3129298/

2.1 Student Registration System

The Student Registration System (SRS) is a database-driven application that is designed to manage and organize academic information in an educational institution. This system is essential in managing and organizing a tremendous amount of data such as student information, course information, lecturer information for courses, and registration information in an organized manner.

The student registration system allows for the management of the relationships between different entities in the system. For instance, a student can be registered for many courses, and a course can have many registered students; thus, it is a many-to-many relationship. On the other hand, a course can be offered by only one department, and a department can offer many courses; thus, it is a one-to-many relationship.

The student registration system also allows for the integrity and security of the data that is stored in the system by providing the following features: authentication, authorization, and backup of the data in the system. The student registration system provides an efficient solution for managing academic information and makes academic decisions better in an educational institution.

2.2 Development Tools

1. MySQL Workbench is used as the main database management tool. This tool is used to design, create, and manage the database structure. Using MySQL Workbench, tables like Student, Course, Department and Enrolment are created, and the relationship between them is established. In addition, it is used to run SQL queries and ensure that data is stored correctly.

2. Draw.io is used to design the Entity-Relationship Diagram (ERD) for the system. Using draw.io, it is possible to visualize the entities, attributes, and relationships in a clear and structured format. This helps to design the database before it is implemented, ensuring that the system is designed according to proper database design principles.

2.3 Key Features and Components

2.3.1 Key Features

The key features of the Student Registration System are focusing on some main entities that manage essential operations in the student registration environment. These entities collectively help order processing more seamlessly, secures administration, technical and academic features. These features have a critical role which enables efficient data flow and customer experience throughout the management process

1. Student Registration and Profile Management

- The student entity provides information's such as subject, studentID and studentPhoneNumber. It helps to checkout smooth, faster assigning throughout the registration process.

2. Course and Curriculum Management

- Allow the Administrator to define the student's academic catalog. This includes course codes, names, descriptions, credits, which department owns the course. It prevents students from enrolling in diploma courses without passing the Foundation courses.

3. Online Self-Enrolment and Course Selection

- Students can look for available sections of the course, check seat availability in real-time,and register for classes. It prevents physical queues and checks the validation of credit limits per semester

4.Faculty and Classroom Scheduling (Timetable)

- Matches lecturer to specific courses and assign them to physical or digital classrooms. It includes conflict – detection to ensure a teacher and or classroom isn't booked twice at the same time.

5.Automated Grading and Academic Tracking

- A feature for teachers to input marks for assignments and exams. The system then automatically calculates GPAs, CGPAs, and determines whether if a student is eligible for the academic probation.

2.3.2 Component

The components of a Student Registration System have 6 entities, which follows database components to operate smoothly in educational system. Student records such as name, age, date of birth, id and gender is recorded to manage the student profiles. StudentID is a unique identifier which allows the student to have a uniquely identifiable that is not the same from other students. For department, it contains the information's like Department_ID, name and office location. These are also organized with categories for easier navigation. The core of the system is the enrolment being enrolled by the students 's course contains details like course name, code, credits and fee.
After that, The Enrolment component is an associative entity that connects the Student and Course components. It is introduced to resolve the many-to-many relationship between students and courses. Besides that, the system also records lecturer contains details like Name, Specialization, Email and Lecturer_ID , which allow them to enrol their course and controls. Finally, the course will be scheduled in the classroom which it will have its own room number, capacity and floor.

2.4 Entity-Relational Diagram (ERD)

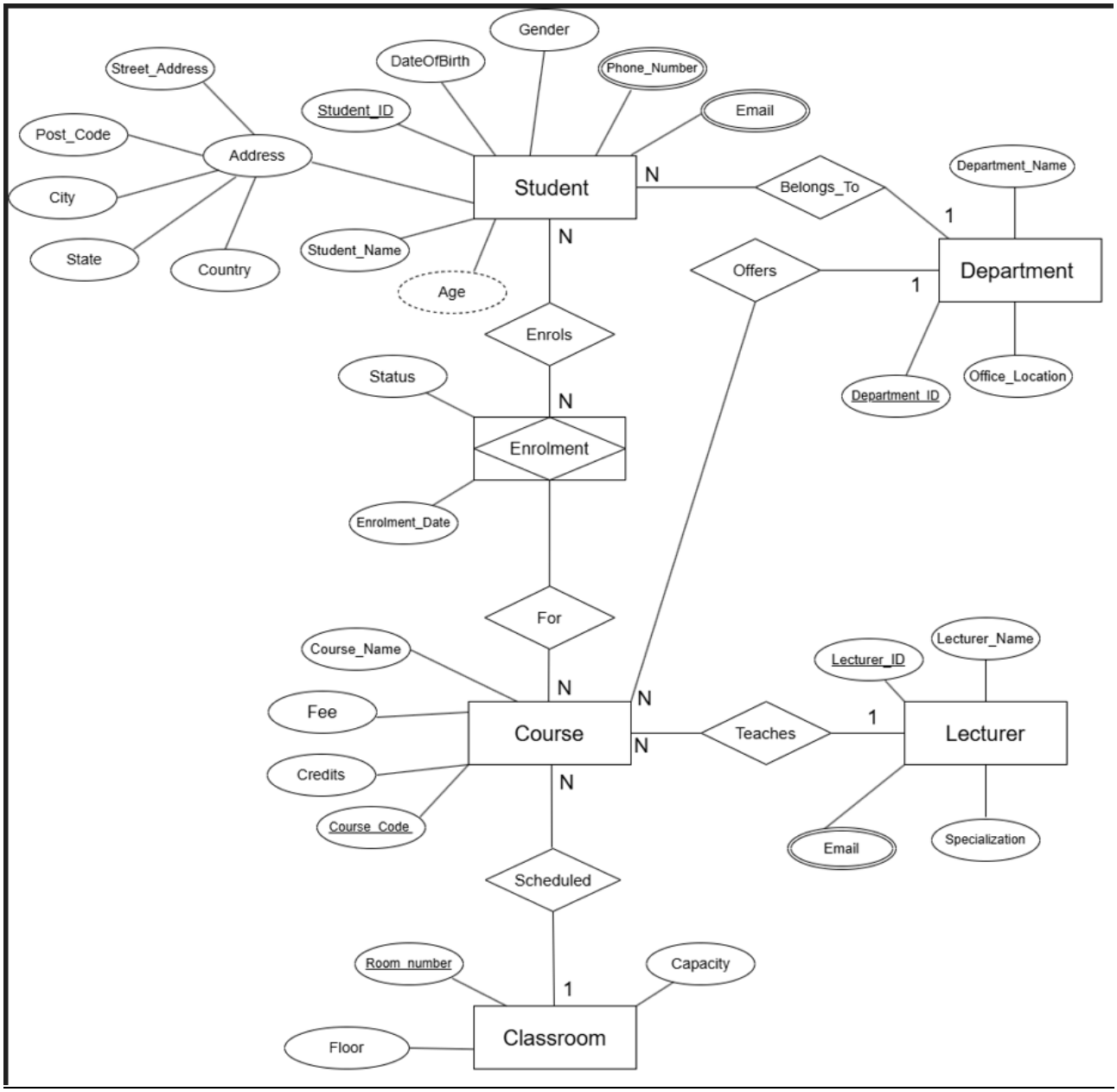


Figure 1 : Chen Notation

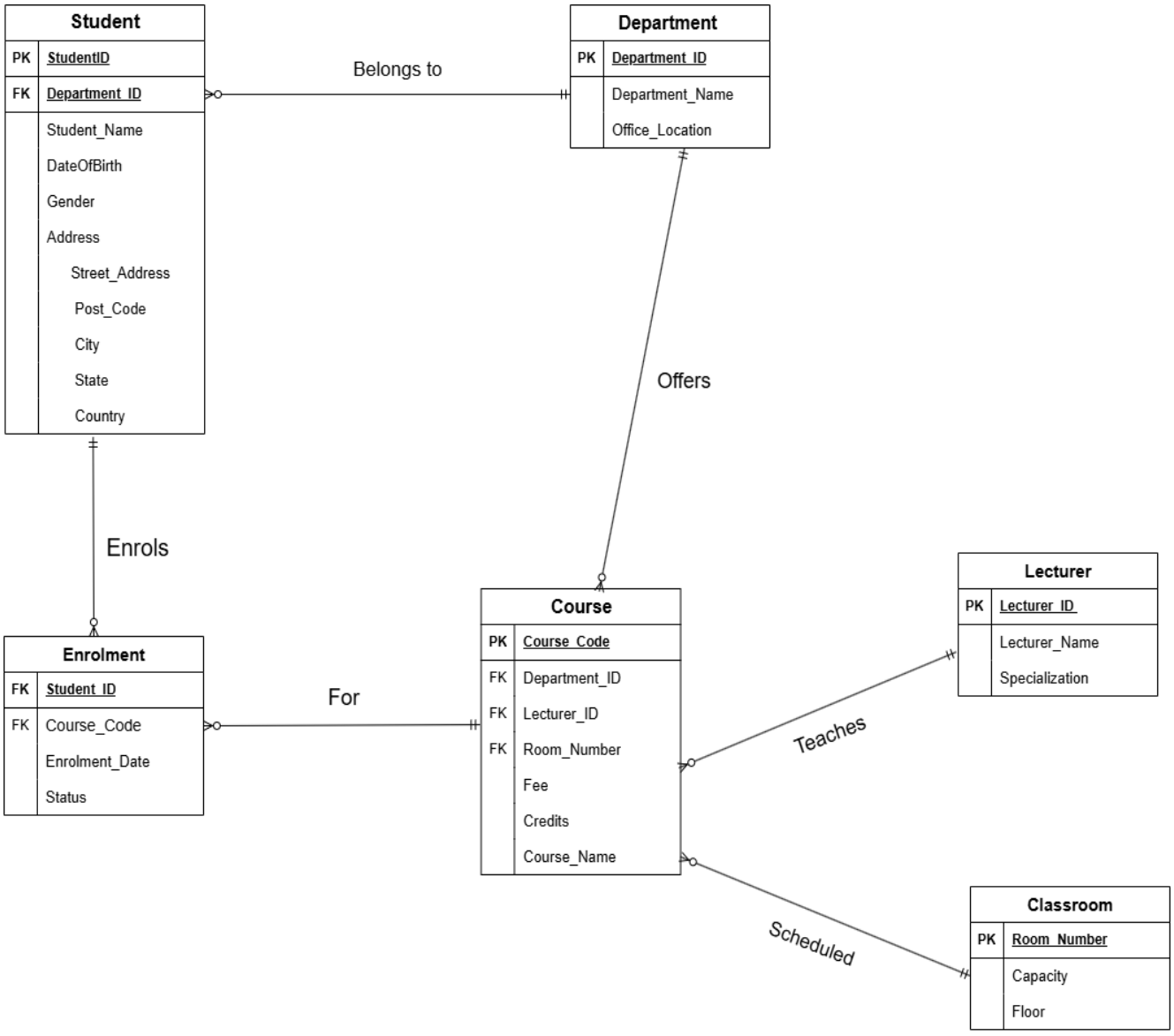


Figure 2 : Crow's foot notation

3.0 Implementation & SQL

3.1 CRUD OPERATION

3.1.1 STUDENT

3.1.1 CREATE

1.

```
DROP TABLE IF EXISTS Student;
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Student_Name VARCHAR(100) NOT NULL,
    DateOfBirth DATE NOT NULL,
    Age INT NULL,
    Gender VARCHAR(10),
    Street_Address VARCHAR(100),
    Post_Code VARCHAR(10),
    City VARCHAR(50),
    State VARCHAR(50),
    Country VARCHAR(50),
    Department_ID VARCHAR(100),
    FOREIGN KEY (Department_ID) REFERENCES Department(Department_ID) ON DELETE CASCADE
```

```
INSERT INTO Student (StudentID, Student_Name, DateOfBirth, Gender, Street_Address, Post_Code, City, State, Country,Department_ID)
VALUES
(1, 'Shong Qien Buo', '2007-08-10', 'Male', '56, Bandar Bukit Tinggi 2, Lorong Batu Nilam 33A', '41299', 'Klang', 'Selangor', 'Malaysia','DIPBUSS'),
(2, 'Irwin Tan', '2007-01-02', 'Male', '89, Bandar Bukit Bayu, Lorong Batu Nilam 22B', '32440', 'Subang', 'Terengganu', 'Malaysia','DIPCOMM'),
(3, 'Calvine Joash', '2007-05-29', 'Male', '34, Bandar Setia Alam, Lorong 44H', '45666', 'Puchong', 'Perak', 'Malaysia','DIPENGR'),
(4, 'Wong Yu Herng', '2007-08-11', 'Male', '66, Bandar Bukit Kepong, Lorong 67D', '56789', 'Kuala Lumpur', 'Perlis', 'Malaysia','DIPHOSP'),
(5, 'Qandeel Asif', '2006-11-06', 'Female', '99, Bandar Bukit Raja, Lorong Batu Nilam 29F', '41430', 'Cheras', 'Negeri Sembilan', 'Malaysia','DIPXDCS');
```

StudentID	Student_Name	DateOfBirth	Age	Gender	Street_Address	Post_Code	City	State	Country	Department_ID
1	Shong Qien Buo	2007-08-10	NULL	Male	56, Bandar Bukit Tinggi 2, Lorong Batu Nilam 33A	41299	Klang	Selangor	Malaysia	DIPBUSS
2	Irwin Tan	2007-01-02	NULL	Male	89, Bandar Bukit Bayu, Lorong Batu Nilam 22B	32440	Subang	Terengganu	Malaysia	DIPCOMM
3	Calvine Joash	2007-05-29	NULL	Male	34, Bandar Setia Alam, Lorong 44H	45666	Puchong	Perak	Malaysia	DIPENGR
4	Wong Yu Herng	2007-08-11	NULL	Male	66, Bandar Bukit Kepong, Lorong 67D	56789	Kuala Lumpur	Perlis	Malaysia	DIPHOSP
5	Qandeel Asif	2006-11-06	NULL	Female	99, Bandar Bukit Raja, Lorong Batu Nilam 29F	41430	Cheras	Negeri Sembilan	Malaysia	DIPXDCS
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 1 : Created Student table

Explanation :

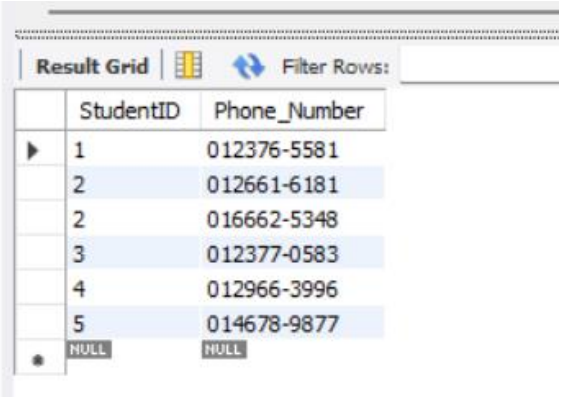
Created a table called Student using syntax CREATE and use INSERT syntax to insert the information such as StudentID(PK), Student_Name , DateOfBirth,Gender , Post_Code , City, State and Country.

- Age is derived from DateOfBirth and may be calculated to ensure accuracy. (derived attributes)
- Street_Address is a composite attributes consist of Post_Code,City,State and Country.

2.

```
DROP TABLE IF EXISTS ContactNumber;
CREATE TABLE ContactNumber(
    StudentID INT NOT NULL,
    Phone_Number VARCHAR(20) NOT NULL,
    PRIMARY KEY (StudentID, Phone_Number),
    FOREIGN KEY (StudentID)
    REFERENCES Student(StudentID)
    ON DELETE CASCADE
);
```

```
INSERT INTO ContactNumber(StudentID,Phone_Number)
VALUES ( 001,'012376-5581'),
       ( 002,'016662-5348'),
       ( 002,'012661-6181'),
       ( 003,'012377-0583'),
       ( 004,'012966-3996'),
       ( 005,'014678-9877');
```



StudentID	Phone_Number
1	012376-5581
2	012661-6181
2	016662-5348
3	012377-0583
4	012966-3996
5	014678-9877
NULL	NULL

Figure 2 : Created Contact Number table

Explanation :

Created a separate table using CREATE syntax for student's contact number because phone number is a multivalued attributes and used INSERT syntax to insert the phone number into the table

3.

```
DROP TABLE IF EXISTS EmailAddress;
CREATE TABLE EmailAddress(
        StudentID INT NOT NULL,
        Email VARCHAR(100),
        PRIMARY KEY(StudentID,Email),
        FOREIGN KEY(StudentID) REFERENCES Student(StudentID) ON DELETE CASCADE
        );
```

```
INSERT INTO EmailAddress(StudentID,Email)
VALUES (001,'jaspershong2006@gmail.com'),
       (001,'M44100768@student.uow.edu.my'),
       (002,'irwin67@gmail.com'),
       (003,'calvine88@gmail.com'),
       (004,'wong23@hotmail.com'),
       (005,'asif123@gmail.com');
```

Result Grid | Filter Rows: | Edit:

StudentID	Email
1	jaspershong2006@gmail.com
1	M44100768@student.uow.edu.my
2	irwin67@gmail.com
3	calvine88@gmail.com
4	wong23@hotmail.com
5	asif123@gmail.com
NULL	NULL

Figure 3 : Created a EmailAddress table

Explanation :

Created a separate table using CREATE syntax for student's email address because email address is a multivalued attributes and used INSERT syntax to insert the email into the table

3.1.1.2 READ

1.

```
SELECT StudentID,Student_Name
FROM Student
WHERE StudentID = 1;
```

	StudentID	Student_Name
▶	1	Shong Qien Buo
*	NULL	NULL

Figure 4 : filtered to find a specific StudentID and Student_Name from Student table

Explanation :

filtered a larger list of students to find a specific individual by their unique ID which is StudentID = 001 . In database terms, this is called a **Projection** (choosing columns) and a **Selection** (choosing rows). Uses WHERE syntax to find the specific data

2.

```
SELECT s.Student_Name, ea.Email
FROM Student s
JOIN EmailAddress ea ON s.StudentID = ea.StudentID
ORDER BY Student_Name ASC;
```

	Student_Name	Email
▶	Calvine Joash	calvine88@gmail.com
	Irwin Tan	irwin67@gmail.com
	Qandeel Asif	asif123@gmail.com
	Shong Qien Buo	jaspershong2006@gmail.com
	Shong Qien Buo	M44100768@student.uow.edu.my
	Wong Yu Herng	wong23@hotmail.com

Explanation :

Uses JOIN syntax to join two separate tables together into a single result based on a common piece of data which is Student_Name and Email and based on ascending orde

3.

```
SELECT
    s.Student_Name,
    cn.Phone_Number,
    ea.Email
    FROM Student s
LEFT JOIN ContactNumber cn ON s.StudentID = cn.StudentID
LEFT JOIN EmailAddress ea ON s.StudentID = ea.StudentID
WHERE s.StudentID = 3;
```

	Student_Name	Phone_Number	Email
▶	Calvine Joash	012377-0583	calvine88@gmail.com

Explanation :

This query creates a **complete profile** for a specific student by pulling data from three different tables at once which is Student ,ContactNumber and EmailAddress table .

3.1.1.3 UPDATE

1.

```
UPDATE Student
SET Street_Address = '12, Jalan Flora, Lorong Batu Nilam 67',
    City = 'Shah Alam',
    Post_Code = '40000'
WHERE StudentID = 1;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

StudentID	Student_Name	DateOfBirth	Age	Gender	Street_Address	Post_Code	City	State	Country
1	Shong Qien Buo	2007-08-10	NULL	Male	12, Jalan Flora, Lorong Batu Nilam 67	40000	Shah Alam	Selangor	Malaysia
2	Irwin Tan	2007-01-02	NULL	Male	89,Bandar Bukit Bayu,Lorong Batu Nilam 22B	32440	Subang	Terengganu	Malaysia
3	Calvine Joash	2007-05-29	NULL	Male	34,Bandar Setia Alam,Lorong 44H	45666	Puchong	Perak	Malaysia
4	Wong Yu Herng	2007-08-11	NULL	Male	66,Bandar Bukit Kepong,Lorong 67D	56789	Kuala Lumpur	Perlis	Malaysia
5	Qandeel Asif	2006-11-06	NULL	Female	99,Bandar Bukit Raja,Lorong Batu Nilam 29F	41430	Cheras	Negeri Sembilan	Malaysia
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation :

Updating Student table Street_Address , City and Post_Code value for student using StudentID = 1 condition.

2.

```
UPDATE EmailAddress
SET Email = 'shong_new2026@gmail.com'
WHERE StudentID = 1 AND Email = 'jaspershong2006@gmail.com';
```

StudentID	Email
1	M44100768@student.uow.edu.my
1	shong_new2026@gmail.com
2	irwin67@gmail.com
3	calvine88@gmail.com
4	wong23@hotmail.com
5	asif123@gmail.com
NULL	NULL

Explanation :

Updating Email Address table ‘s Email value for Student with condition of StudentID = 1 and Email = ‘jaspershong2003@gmail.com’.

3.1.1.4 DELETE

1.

```
DELETE FROM EmailAddress
WHERE StudentID = 1 AND Email = 'jaspershong2006@gmail.com'
```

StudentID	Email
1	M44100768@student.uow.edu.my
2	irwin67@gmail.com
3	calvine88@gmail.com
4	wong23@hotmail.com
5	asif123@gmail.com
NULL	NULL

Explanation :

Deleting his personal email but keep his student email for StudentID = 1 and Email = 'jaspershong2006@gmail.com'.

2.

```
DELETE FROM Student
WHERE StudentID = 3;
```

StudentID	Student_Name	DateOfBirth	Age	Gender	Street_Address	Post_Code	City	State	Country
1	Shong Qien Buo	2007-08-10	NULL	Male	12, Jalan Flora, Lorong Batu Nilam 67	40000	Shah Alam	Selangor	Malaysia
2	Irwin Tan	2007-01-02	NULL	Male	89,Bandar Bukit Bayu,Lorong Batu Nilam 22B	32440	Subang	Terengganu	Malaysia
4	Wong Yu Herng	2007-08-11	NULL	Male	66,Bandar Bukit Kepong,Lorong 67D	56789	Kuala Lumpur	Perlis	Malaysia
5	Qandeel Asif	2006-11-06	NULL	Female	99,Bandar Bukit Raja,Lorong Batu Nilam 29F	41430	Cheras	Negeri Sembilan	Malaysia
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation :

Deleting Student 's information for StudentID = 3 because the student have drop out from the University.

3.1.2 Course

3.1.2.1 CREATE

1.

```
CREATE TABLE Course (
    Course_Code VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Credits INT,
    Fee DECIMAL(10, 2),
    Department_ID VARCHAR(100),
    Room_Number VARCHAR(10),
    Lecturer_ID VARCHAR(10),
    FOREIGN KEY (Department_ID) REFERENCES Department(Department_ID),
    FOREIGN KEY (Room_Number) REFERENCES Classroom(Room_Number),
    FOREIGN KEY (Lecturer_ID) REFERENCES Lecturer(Lecturer_ID)
);
```

```
INSERT INTO Course (Course_Code, Name, Credits, Fee,Department_ID,Room_Number,Lecturer_ID)
VALUES ('BUSS1013', 'Principles Of Marketing', 3, 900.00,'DIPBUSS','R001','L001'),
       ('COMM2014', 'Mass Communication Theory', 2, 500.00,'DIPCOMM','R002','L002'),
       ('ENGR1014', 'Introduction to Mechanical Engineering', 3, 1250.00,'DIPENGR','R003','L003'),
       ('HOSP1023', 'Introduction to Hospitality', 2, 950.00,'DIPHOSP','R004','L004'),
       ('XDCS1054N', 'Database Systems', 4, 1200.00,'DIPXDCS','R005','L005');
```

Course_Code	Name	Credits	Fee	Department_ID	Room_Number	Lecturer_ID
BUSS1013	Principles Of Marketing	3	900.00	DIPBUSS	R001	L001
COMM2014	Mass Communication Theory	2	500.00	DIPCOMM	R002	L002
ENGR1014	Introduction to Mechanical Engineering	3	1250.00	DIPENGR	R003	L003
HOSP1023	Introduction to Hospitality	2	950.00	DIPHOSP	R004	L004
XDCS1054N	Database Systems	4	1200.00	DIPXDCS	R005	L005
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation :

Created a table called Course using CREATE syntax and used INSERT syntax to insert all the information related to Course such as Course_Code(PK) , Course_Name,Credits , Fee and Department_ID (FK) ,Room_Number(FK),Lecturer_ID(FK).

3.1.2.2 READ

1.

```
SELECT *
FROM Course
WHERE Course_Code = 'COMM2014';
```

	Course_Code	Course_Name	Credits	Fee	DeptID
	COMM2014	Mass Communication Theory	2	500.00	DIPCOMM
▶*		NULL	NULL	NULL	NULL

Explanation :

This query retrieves all information for a specific course from the **Course** table WHERE Course_Code(PK) = 'COMM2014'.

2.

```
SELECT COUNT(*) AS Total_Courses
FROM Course;
```

	Total_Courses
▶	5

Explanation :

This query counts the total number of courses in the **Course** table which is 5 total courses.

3.1.2.3 UPDATE

1.

```
UPDATE Course
SET Course_Name = 'Principles Of Management' ,
 Course_Code ='BUSS2013',
 Credits = 4,
 Fee = 1350.00
WHERE Course_Code = 'BUSS2013';
```

Course_Code	Course_Name	Credits	Fee
BUSS2013	Principles Of Management	4	1350.00
COMM2014	Mass Communication Theory	2	500.00
ENGR1014	Introduction to Mechanical Engineering	3	1000.00

Explanation :

This query update the Course_Name = Principles of Management , Credits = 4 and Fee = 1350.00 by referring to Course_Code = 'BUSS2013'.

2.

```
UPDATE Course
SET Fee = Fee * 0.80
WHERE Course_Code ='ENGR1014';
```

Result Grid | Filter Rows: | Edit: | Expo

Course_Code	Name	Credits	Fee	D
BUSS1013	Principles Of Marketing	3	900.00	D
COMM2014	Mass Communication Theory	2	500.00	D
ENGR1014	Introduction to Mechanical Engineering	3	1000.00	D

Explanation:

This query updates the Course table by reducing the fee of the course with Course_Code 'ENGR1014' by 20% which is now cost 1000.00. (1250.00 * 0.80)

3.1.2.4 DELETE

1.

```
DELETE FROM Course
WHERE Credits < 3;
```

Course_Code	Course_Name	Credits	Fee	Department_ID	Room_Number	Lecturer_ID
BUSS2013	Principles Of Management	4	1350.00	DIPBUSS	R001	L001
ENGR1014	Introduction to Mechanical Engineering	3	1000.00	DIPENGR	R003	L003
XDCS1054N	Database Systems	4	1200.00	DIPXDCS	R005	L005
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation :

Deletes all courses from the Course table that have fewer than 3 credits which is Mass Communication Theory and Introduction to Hospitality.

2.

```
DELETE FROM Course
WHERE Course_Code IN ('ENGR1014', 'XDCS1054N');
```

Course_Code	Course_Name	Credits	Fee	DeptID
BUSS2013	Principles Of Management	4	1350.00	DIPBUSS
COMM2014	Mass Communication Theory	2	500.00	DIPCOMM
HOSP1023	Introduction to Hospitality	2	950.00	DIPHOSP
NULL	NULL	NULL	NULL	NULL

Explanation

Deletes the courses with Course_Code = ENGR1014 and XDCS1054N from the Course table.

3.1.3 DEPARTMENT

3.1.3.1 CREATE

1.

```
CREATE TABLE Department (
    Department_ID VARCHAR(10) PRIMARY KEY,
    Department_name VARCHAR(100) NOT NULL,
    Office_Location VARCHAR(100) NOT NULL
);
```

```
INSERT INTO Department (Department_ID, Department_name, Office_Location)
VALUES
('DIPBUSS', 'School Of Business', 'Block A'),
('DIPCOMM', 'School Of Communication', 'Block B'),
('DIPENGR', 'School Of Engineering', 'Block C'),
('DIPHOSP', 'School Of Hospitality', 'Block D'),
('DIPXDCS', 'School Of Computer Science', 'Block E');
```

Department_ID	Department_name	Office_Location
DIPBUSS	School Of Business	Block A
DIPCOMM	School Of Communication	Block B
DIPENGR	School Of Engineering	Block C
DIPHOSP	School Of Hospitality	Block D
DIPXDCS	School Of Computer Science	Block E
NULL	NULL	NULL

Explanation :

Created a table called Department using syntax CREATE and use INSERT syntax to insert the information such as Department_ID (PK), Department_Name and Office_Location.
It allows the system to store and manage different academic departments.

3.1.3.2 READ

1.

```
SELECT *
FROM Department
WHERE Department_ID = 'DIPBUSS';
```

Department_ID	Department_name	Office_Location
DIPBUSS	School Of Business	Block A
NULL	NULL	NULL

Explanation :

This queries retrieve department data from the database which is retrieves a specific department based on its Department_ID = 'DIPBUSS'.

2.

```
SELECT s.Student_name, d.Department_name
FROM Student s
JOIN Department d
ON s.Department_ID = d.Department_ID
WHERE d.department_name = 'School of Business';
```

Student_name	Department_name
Shong Qien Buo	School Of Business

Explanation :

The JOIN operation is used to combine data from the Student and Department tables based on the Department_ID. This allows the system to display the specific student along with the corresponding Department_name.

3.1.1.3 UPDATE

1

```
UPDATE Department
SET Office_Location = 'Block C'
WHERE Department_ID = 'DIPBUSS';
```

Department_ID	Department_name	Office_Location
DIPBUSS	School Of Business	Block C
DIPCOMM	School Of Communication	Block B
DIPENGR	School Of Engineering	Block C
DIPHOSP	School Of Hospitality	Block D
DIPXDCS	School Of Computer Science	Block E
NULL	NULL	NULL

Explanation :

This operation updates existing department information, ensuring that the database remains accurate and up to date which update the Office_Location to 'Block C' WHERE Department_ID = 'DIPBUSS'

2.

```
UPDATE Department
SET department_name = 'School of Engineering Technology'
WHERE Department_ID = 'DIPENGR';
```

Department_ID	Department_name	Office_Location
DIPBUSS	School Of Business	Block C
DIPCOMM	School Of Communication	Block B
DIPENGR	School of Engineering Technology	Block C
DIPHOSP	School Of Hospitality	Block D
DIPXDCS	School Of Computer Science	Block E
NULL	NULL	NULL

Explanation :

This query updates the name of the department, allowing the system to reflect changes such as restructuring of academic departments which UPDATE department_name from School of Engineering to School of Engineering Technology WHERE Department_ID = 'DIPENGR'.

3.1.3.4 DELETE

1.

```
DELETE FROM Department
WHERE Department_ID = 'DIPCOMM';
```

Department_ID	Department_name	Office_Location
DIPBUSS	School Of Business	Block C
DIPENGR	School of Engineering Technology	Block C
DIPHOSP	School Of Hospitality	Block D
DIPXDCS	School Of Computer Science	Block E
NULL	NULL	NULL

Explanation :

This operation removes a department record from the database when it is no longer required which is Departmen_ID = ‘DIPCOMM’.

2.

```
DELETE FROM Department
WHERE department_name = 'School of Hospitality'
AND Office_Location = 'Block D';
```

Department_ID	Department_name	Office_Location
DIPBUSS	School Of Business	Block C
DIPENGR	School of Engineering Technology	Block C
DIPXDCS	School Of Computer Science	Block E
NULL	NULL	NULL

Explanation :

This query removes a specific department by combining multiple conditions, reducing the risk of deleting unintended records WHERE department_name = ‘School of Hospitality’ AND Office_Location = ‘Block D’.

3.1.4 ENROLLMENT

3.1.4.1 CREATE

```
DROP TABLE IF EXISTS Enrolment;
CREATE TABLE Enrolment (
    StudentID INT,
    Course_Code VARCHAR(10),
    Enrolment_Date DATE,
    Status VARCHAR(20),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (Course_Code) REFERENCES Course(Course_Code)
    ON DELETE CASCADE
);
```

```
INSERT INTO Enrolment (StudentID, Course_Code, Enrolment_Date, Status)
VALUES ( 1,'BUSS2013','2026-03-20','ACTIVE'),
       ( 2,'COMM2014','2025-04-12','ACTIVE'),
       ( 3,'ENGR1014','2024-02-16','ACTIVE'),
       (4,'HOSP1023','2026-09-24','ACTIVE'),
       (5,'XDCS1054N','2026-04-01','PENDING');
```

	StudentID	Course_Code	Enrolment_Date	Status
▶	1	BUSS2013	2026-03-20	ACTIVE
	2	COMM2014	2025-04-12	ACTIVE
	3	ENGR1014	2024-02-16	ACTIVE
	4	HOSP1023	2026-09-24	ACTIVE
	5	XDCS1054N	2026-04-01	PENDING

Explanation :

Create a table called Enrolment using CREATE syntax and use INSERT syntax to inserts new enrolment records into the Enrolment table. It is used to record which students have enrolled in which specific course.

3.1.4.2 READ

1.

```
SELECT * FROM Enrolment;
```

Explanation :

This query syntax is to display all the enrolment data using SELECT * FROM syntax.

2.

```
SELECT
    s.Student_Name,
    s.StudentID,
    c.Course_Name,
    e.Enrolment_Date,
    e.Status,
    d.Department_Name
FROM Enrolment e
JOIN Student s ON e.StudentID = s.StudentID
JOIN Course c ON e.Course_Code = c.Course_Code
JOIN Department d ON s.Department_ID = d.Department_ID;
```

Student_Name	StudentID	Course_Name	Enrolment_Date	Status	Department_Name
Shong Qien Buo	1	Principles Of Management	2026-03-20	ACTIVE	School Of Business
Irwin Tan	2	Mass Communication Theory	2025-04-12	ACTIVE	School Of Communication
Calvine Joash	3	Introduction to Mechanical Engineering	2024-02-16	ACTIVE	School of Engineering Technology
Wong Yu Herng	4	Introduction to Hospitality	2026-09-24	ACTIVE	School Of Hospitality
Qandeel Asif	5	Database Systems	2026-04-01	PENDING	School Of Computer Science

Explanation :

This query joins the Enrolment, Student, Course, and Department tables to display complete information about student enrolments, including student details, course information, enrolment date, and department name.

3.1.4.3 UPDATE

1.

```
UPDATE Enrolment
SET Status = 'ACTIVE'
WHERE StudentID = 5;
```

StudentID	Course_Code	Enrolment_Date	Status
1	BUSS2013	2026-03-20	ACTIVE
2	COMM2014	2025-04-12	ACTIVE
3	ENGR1014	2024-02-16	ACTIVE
4	HOSP1023	2026-09-24	ACTIVE
5	XDCS1054N	2026-04-01	ACTIVE

Explanation :

This query syntax updates the existing enrolment details. This ensures that the data is accurate and up to date which is using SET syntax to set the status “Pending” to “Active” WHERE StudentID = 5.

2.

```
UPDATE Enrolment
SET Enrolment_Date = '2020-06-07'
WHERE StudentID = 2;
```

StudentID	Course_Code	Enrolment_Date	Status
1	BUSS2013	2026-03-20	ACTIVE
2	COMM2014	2020-06-07	ACTIVE
3	ENGR1014	2024-02-16	ACTIVE
4	HOSP1023	2026-09-24	ACTIVE
5	XDCS1054N	2026-04-01	ACTIVE

Explanation :

This query syntax updates the existing enrolment details. This ensures that the data is accurate and up to date which is using SET syntax to set Enrolment_Date from ‘2019-03-12’ to ‘2020-06-07’ WHERE StudentID = 2.

3.1.4.4 DELETE

1.

```
DELETE FROM Enrolment
WHERE StudentID = 3;
```

StudentID	Course_Code	Enrolment_Date	Status
1	BUSS2013	2026-03-20	ACTIVE
2	COMM2014	2020-06-07	ACTIVE
4	HOSP1023	2026-09-24	ACTIVE
5	XDCS1054N	2026-04-01	ACTIVE

Explanation :

The query syntax is removing enrolment records from the table. This may be used when an enrolment record is cancelled or the student already dropout from the school which is StudentID = 3.

2.

```
DELETE FROM Enrolment
WHERE Status = 'Pending';
```

StudentID	Course_Code	Enrolment_Date	Status
2	COMM2014	2020-06-07	ACTIVE
5	XDCS1054N	2026-04-01	ACTIVE

Explanation :

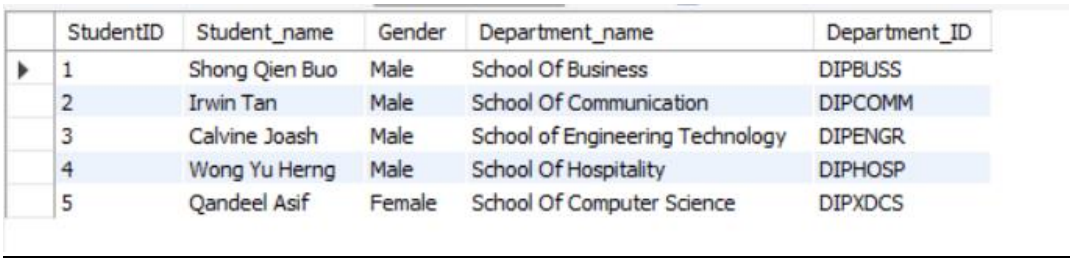
The query syntax is removing enrolment records from the table. This may be used when an enrolment status is pending and the students decided to not continue their studies which is StudentID = 1 and 4.

3.2 ADVANCED SQL OBJECTS

3.2.1 VIEW

3.2.1.1 Student and Department View

```
DROP VIEW IF EXISTS StudentDepartmentView;
CREATE VIEW StudentDepartmentView AS
SELECT s.StudentID, s.Student_name,s.Gender, d.Department_name,d.Department_ID
FROM Student s
JOIN Department d
ON s.Department_ID = d.Department_ID;
```



StudentID	Student_name	Gender	Department_name	Department_ID
1	Shong Qien Buo	Male	School Of Business	DIPBUSS
2	Irwin Tan	Male	School Of Communication	DIPCOMM
3	Calvine Joash	Male	School of Engineering Technology	DIPENGR
4	Wong Yu Herng	Male	School Of Hospitality	DIPHOSP
5	Qandeel Asif	Female	School Of Computer Science	DIPXDCS

Explanation :

The image stated above is to shows SQL query to create a view named StudentDepartmentView. The StudentDepartmentView is created to display student details such as StudentID,Student_Name and Gender along with their corresponding department information include Department_Name and Department_ID by joining the Student and Department tables using Department_ID. It simplifies data retrieval and improves query efficiency. This view is useful in the Student Registration System because it allows users to easily retrieve student information together with their department details without repeatedly writing JOIN statements.It also can improve readability and usability of the database.It can simplify complex queries.

3.2.2 STORED PROCEDURE

3.2.2.1 Update Course Fee

```
DELIMITER //

CREATE PROCEDURE UpdateCourseDetails(
    IN courseCode VARCHAR(10),
    IN newRoom VARCHAR(10),
    IN newFee DECIMAL(10,2)
)
BEGIN
    UPDATE Course
    SET Room_Number = newRoom,
        Fee = newFee
    WHERE Course_Code = courseCode;
END //

DELIMITER ;
```

```
CALL UpdateCourseDetails('COMM2014', 'R004',1000.00);
```

Course_Code	Course_Name	Credits	Fee	Department_ID	Room_Number	Lecturer_ID
BUSS2013	Principles Of Management	4	1350.00	DIPBUSS	R001	L001
COMM2014	Mass Communication Theory	2	1000.00	DIPCOMM	R004	L002
ENGR1014	Introduction to Mechanical Engineering	3	1000.00	DIPENGR	R003	L003
HOSP1023	Introduction to Hospitality	2	950.00	DIPHOSP	R004	L004

Explanation :

The image stated above shows the SQL query and output of the stored procedure UpdateCourseDetails.The purpose of this stored procedure is to modify multiple attributes of a course in a single execution.It accepts three input which is courseCode is to identifies the course,newRoom is to updates the room number assigned and newFee is to updates the course fee.It can improve efficiency and reducing the need for multiple SQL statements.

3.2.2.1 Add New Student

```
DELIMITER //

CREATE PROCEDURE AddNewStudent(
    IN p_studentid VARCHAR(10),
    IN p_name VARCHAR(100),
    IN p_dob DATE,
    IN p_gender VARCHAR(10),
    IN p_street_address VARCHAR(100),
    IN p_post_code VARCHAR(10),
    IN p_city VARCHAR(50),
    IN p_state VARCHAR(50),
    IN p_country VARCHAR(50),
    IN p_dept_id VARCHAR(10)
)
```

```
BEGIN
    INSERT INTO Student
    (StudentID, Student_Name, DateOfBirth, Gender,
     Street_Address, Post_Code, City, State, Country, Department_ID)

    VALUES
    (p_studentid, p_name, p_dob, p_gender,
     p_street_address, p_post_code, p_city, p_state, p_country, p_dept_id);
END //

DELIMITER ;
```

```
CALL AddNewStudent(
    6,
    'Sheva Amir',
    '2005-07-20',
    'Female',
    '67,Bandar Puchong,Lorong 67C',
    '42345',
    'Klang',
    'Selangor',
    'Malaysia',
    'DIPCOMM'
);
```

5	Qandeel Asif	2006-11-06	NULL	Female	99, Bandar Bukit Raja, Lorong Batu Nilam 29F	41430	Cheras	Negeri Sembilan	Malaysia	DIPXDCS
6	Sheva Amir	2005-07-20	NULL	Female	67,Bandar Puchong,Lorong 67C	42345	Klang	Selangor	Malaysia	DIPCOMM
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation:

The image stated above shows the SQL query and output of the stored procedure AddNewStudent .The purpose of the stored procedure is to insert a new student record along with complete address information.It accepts multiple input include personal details (ID, name, DOB, gender), address details (street, postcode, city, state, country) and department ID.These query allows to insert all the student information in one execution.

3.2.3 TRIGGER

3.2.3.1 Validate Course Fee

```
DELIMITER //

CREATE TRIGGER ValidateCourseFee
BEFORE INSERT ON Course
FOR EACH ROW
BEGIN
    IF NEW.fee < 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Course fee must be at least 100';
    END IF;
END //

DELIMITER ;
```

```
INSERT INTO Course (Course_Code, Course_Name, Credits, Fee,Department_ID,Room_Number,Lecturer_ID)
VALUES ('COMM456','Introduction to Public Relation',2,20.00,'DIPCOMM','R006','L006');
```

```
54  15:47:06  INSERT INTO Course (Course_Code, Course_Name, Credits, Fee,Department_ID,Room_Number,Lecturer_ID)...  Error Code: 1644. Course fee must be at least 100
```

Explanation :

The image stated above shows the SQL Query and output of the TRIGGER ValidateCourseFee.The purpose of the TRIGGER is to prevents invalid or unrealistic course fees from being inserted.If the fee is < 100 ,the system will displays a text message saying “Course fee must be at least 100”. The benefits of this is to ensure validation of the input is correct before confirmation without any manual intervention.It also can automatically react to data changes.

3.2.3 FUNCTION

3.2.3.1 Calculated Discounted Fee

```
DELIMITER //

CREATE FUNCTION CalculateDiscountedFee(
    originalFee DECIMAL(10,2),
    discountPercent DECIMAL(5,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE discountedFee DECIMAL(10,2);

    SET discountedFee = originalFee - (originalFee * discountPercent / 100);

    RETURN discountedFee;
END //

DELIMITER ;
```

```
SELECT
    Course_Code,
    Course_Name,
    CONCAT(20,'%') AS Discount_Percentage,
    Fee AS Original_Fee,
    CalculateDiscountedFee(fee, 20) AS Discounted_Price
FROM Course
WHERE Course_Code = 'COMM2014';
```

	Course_Code	Course_Name	Discount_Percentage	Original_Fee	Discounted_Price
▶	COMM2014	Mass Communication Theory	20%	1000.00	800.00

Explanation :

The image stated above shows the SQL query and the output of the FUNCTION CalculateDiscountedFee.The purpose of the FUNCTION is to calculate the total fee after applying a discount to a course fee. This supports easier financial calculations in the database system. It accepts two input which is originalFee = the original course fee and discountPercent = the percentage of discount to be applied. The function calculates the discounted fee using this formula:

Discounted Fee = Original Fee - (Original Fee × Discount Percentage / 100)
This function is useful because it avoids repeated manual calculations and allow the system to retrieve the discounted price within the SQL query.Its also return computed values.

4.0 Real-Life Examples

In today's education industry, Database Management Systems (DBMS) have been used in supporting numerous academic or educational systems. For instance, educational institutions such as schools, colleges, or even universities have been using database systems in effectively managing numerous data.

One example is in university student management systems. A university provides students with the opportunity to access their own personal information, enrol in certain courses, check class schedules, or even check their examination results. All of this is being stored in a database system. Therefore, all of the students' information is being stored in a database system, which is accurate, up-to-date, and easily accessible.

Another example is in course registration systems. A database management system is used in supporting numerous academic or educational systems. For instance, educational institutions have been using database systems in effectively managing numerous data.

Generally, it is safe to conclude that the real-life education systems are good examples that illustrate the significant role DBMS plays in improving efficiency and accuracy in handling data in educational institutions.

Almehmes, I. A. H. (2014). *Design and implement of database student information management system in College of Medicine – University of Diyala* (Unpublished master's thesis). Çankaya University. http://hdl.handle.net/20.500.12416/324

4.1 Companies / Institutions Using DBMS in Education

Many educational institutions worldwide use Database Management Systems (DBMS) to manage their educational activities in the most efficient manner. Universities and colleges are using database systems to manage their activities efficiently.

For example, universities like University of Wollongong (UOW) are using student information systems, which are supported by centralized databases. With the support of these systems, universities are able to manage student enrolment activities in the most efficient manner.

In addition, universities are using online learning systems, which are supported by database systems. With the support of these systems, universities are able to store their critical data, which includes course materials, assignments, and grades.

Additionally, most institutions have an integrated campus management system in place. In this system, several activities such as student registration, finance, and academic records are all linked and stored in one database. This helps in improving integration and coordination among various departments in the institution. All the data is consistent and updated. Therefore, these institutions show the significance of the DBMS in modern education.

4.2 Role of DBMS in Operations

A Database Management System (DBMS) helps in the efficient and successful operation of educational institutions. It manages a large amount of information relating to education in a very efficient manner.

Firstly, DBMS helps in the efficient management of data. It does this by organizing the information in a database. Data relating to students, courses, lecturers, and so on can be stored in a database. It helps in avoiding duplication of information and inconsistencies in the information stored in different sections of the educational institution.

Secondly, DBMS helps in efficient operation. It does this by ensuring that tasks are completed in a very efficient manner. Course registration, updating student information, and generating reports are all efficiently done.

In addition to this, the data is made more secure with the restriction of access to authorized users only. The data that needs to be kept confidential, such as the personal details and academic results of the students, is made more secure with the implementation of authentication and access control. The backup and recovery options also prevent data loss in the event of system failure.

Further, the DBMS provides the facility of proper storage of student records. This helps the educational institutions in maintaining complete and accurate records of the students over time. The records can be retrieved when needed for academic purposes.

In conclusion, the DBMS plays a vital role in the functioning of educational institutions.

5.0 Suggestions for Improvement

5.1 Current Limitation and Challenges

In the modern era, the educational institutions have been starting to depend on digital Student Registration Systems to manage administration and academic workflows effectively. However, the system could have several limitations and challenges that may affect its performance and data management. The research examines three primary issues that may face in the use of Student Registration System in education, which are data inconsistency, security concerns and maintenance.

The first major limitation for the system is data consistency and redundancy. According to The Design and Implementation of Student Academic Record Management System. (2011). *Airiti Library*. https://www.airitilibrary.com/Article/Detail?DocID=20407467-201108-201411110030-201411110030-707-712, inconsistencies such as conflating student information or duplicate enrolment records can affect system reliability and institutional decision-making. In education environments, redundant data can lead to errors in grading calculation or transcript generation, which can compromise student's academic progress.

Another crucial limitation is security and privacy concerns. According to N/A, secure database systems must integrate advanced encryption and strict access controls to reduce risks related to unauthorized access and cyberattacks. Because these systems store sensitive personal and financial information. A failure to implement risk management measures will result in data loss and harm the education reputation.

Maintenance is also critical for the system's long-term performance. Implementing scalable database and continuous performance tuning helps prevent overloads and downtime during heavy traffic periods, such as the first day of course registration. Without resource planning and performance adjustment, the system could experience system timeouts and data loss when thousands of students attempt to access the database simultaneously, causing the system to be failure.

Despite the advantages of using a Student Registration System in managing enrolment, grading, and course data, several limitations still hinder its full effectiveness. Issues such as data inconsistency, security vulnerabilities, and maintenance require continuous system optimization. With proper implementation, the system can operate more reliably and support efficient, data-driven academic operations.

5.2 Proposed Enhancement or Innovations

The Student Registration System developed in the project has introduced the capabilities of rational database using MYSQL, several strategic enhancements can be implemented to optimize the performance, security, and utility of a system. This section provides three key improvements: adoption cloud-based infrastructure, the integration of Artificial Intelligence for academic advising, and the implementation of advanced cybersecurity.

An area of improvement for the system could be cloud-based database. According to **Building Course Registration Project with Azure SQL Database**. (2024). *Azure SQL Dev Corner* (Microsoft). https://devblogs.microsoft.com/azure-sql/course-registration-project-with-azure-sql-database/,using some of the cloud services like AWS or Microsoft Azure allows institutions to implement "Elastic Scaling," which can increase server capacity during high-traffic registration windows. This can eliminate the system downtime and timeouts typically associated with systems, ensuring a clear experience for students even during peak load periods.

Furthermore, the use of Data Analytic**s** can significantly enhance the academic support component of the system. As noted by the references , AI modules can analyse a student's past performance to provide course recommendations and "Early Warning" alerts for those who are at risk of academic prohibition. By transforming the database from a long storage unit into an analytical tool, institutions can improve student retention and provide more targeted academic counselling.

In conclusion, the Student Registration System has introduced most useful functions for education operations in a relational database system. However, there could be further improvements added to improve the overall design. By including constraints to academic record and automating enrolment, its efficiency increases as it provides more accurate to educators that supports their decision making.

6.0 REFERENCES

1. Mukul, E., & Büyüközkan, G. (2023). *Digital transformation in education: A systematic review of education 4.0*. **Technological Forecasting and Social Change, 194**, 122664. https://doi.org/10.1016/j.techfore.2023.122664

2. **Anika, A., & Kumar, D. (2023).** *Streamlining Student Enrollment: University's College Registration System*. Rawat Prakashan. A related chapter similarly describing manual processes as "prone to mistakes" https://lrcdrs.bennett.edu.in/items/6fc43717-5acf-43e4-8e21-ba58d181b9d0

3. **Kaka, A. R. (2023, June 17).** *Unleashing the potential of cloud databases in education; enhancing collaboration, accessibility*. The Financial Express. https://www.financialexpress.com/jobs-career/education-unleashing-the-potential-of-cloud-databases-in-education-enhancing-collaboration-accessibility-3129298/

4. **Almehmes, I. A. H. (2014).** *Design and implement of database student information management system in College of Medicine – University of Diyala* (Unpublished master's thesis). Çankaya University. http://hdl.handle.net/20.500.12416/324

5. The Design and Implementation of Student Academic Record Management System. (2011). *Airiti Library*. https://www.airitilibrary.com/Article/Detail?DocID=20407467-201108-201411110030-201411110030-707-712

6. **Building Course Registration Project with Azure SQL Database**. (2024). *Azure SQL Dev Corner* (Microsoft). https://devblogs.microsoft.com/azure-sql/course-registration-project-with-azure-sql-database/